

Методичка: репозиторий Git для Linux-конфигураций и Bash-сценариев

Темы: Git, working tree, staging area, commits, diff, log, branches, .gitignore, секреты, инфраструктурный репозиторий

ОС сервера: Debian Server 12/13 или Ubuntu Server 22.04/24.04 LTS

ОС клиента: Debian/Ubuntu или Windows 10/11

Среда: VirtualBox

Формат: самостоятельная практическая работа

Ориентировочное время: 4 академических часа

1. Что настроим

В этой работе настраиваем часть Linux-инфраструктуры организации.

Команды выполняем на Debian/Ubuntu, а результат проверяем локально и с клиента.

Используем узлы: linux01.

Главная идея работы:

Git считается настроенным не тогда, когда создан каталог `.git`, а тогда, когда история понятна, изменения проверяются через diff, секреты исключены, а нужную версию можно восстановить.

2. Схема учебного стенда

Узел	Роль	ОС	Сетевые адаптеры	IP-адрес
linux01	основной сервер	Debian/Ubuntu Server	NAT + внутренняя сеть	192.168.10.10/24
client01	клиент проверки	Debian/Ubuntu или Windows	внутренняя сеть	192.168.10.20/24
srv02	дополнительный сервер	Debian/Ubuntu Server	NAT + внутренняя сеть	192.168.10.30/24

```
VirtualBox NAT
|
linux01 ---- внутренняя сеть linux-lab ---- client01/srv02
```

NAT нужен для установки пакетов. Учебные сервисы проверяем во внутренней сети. Если используем имена, заранее добавляем DNS или `/etc/hosts` и проверяем `getent hosts`.

3. Необходимые ресурсы и предварительные условия

Нужны:

- установленный пакет `git`;

- каталог `~/linux-infrastructure`;
- учебные копии конфигураций без секретов;
- сценарий `server-health.sh` из предыдущей работы или простой учебный скрипт.

4. Подготовка виртуальных машин

```
hostnamectl  
ip -br link  
ip -br address  
ip route  
resolvectl status
```

Добавляем имена в `/etc/hosts`, если DNS ещё нет:

```
sudo nano /etc/hosts  
  
192.168.10.10 linux01.company.test linux01  
192.168.10.20 client01.company.test client01  
192.168.10.30 srv02.company.test srv02
```

Проверяем:

```
getent hosts linux01.company.test  
ping -c 3 client01.company.test
```

Самопроверка этапа

- ВМ включены;
- NAT доступен для установки пакетов;
- внутренняя сеть работает;
- имена, используемые в командах, разрешаются.

Если результат отличается от ожидаемого, дальше пока не продолжаем. Сначала проверяем этот этап.

5. Адресный план или план ресурсов

Ресурс	Значение
Основной сервер	linux01, 192.168.10.10
Клиент	client01, 192.168.10.20
Дополнительный сервер	srv02, 192.168.10.30
Учебный домен	company.test
Рабочие каталоги	по сценарию

План работы:

- создать структуру репозитория;
- настроить `.gitignore`;
- добавить конфигурации, сценарии и документацию;
- выполнить commits с понятными сообщениями;
- посмотреть `diff`, `log` и восстановить версию;
- создать ветку, внести изменение и объединить;
- проверить отсутствие секретов.

6. Исходная проверка

```
cat /etc/os-release
hostnamectl
ip -br address
ip route
systemctl --failed
```

```
ss -lntup
```

Самопроверка этапа

- ОС определена;
- сеть работает;
- нет неожиданных failed-служб;
- исходное состояние записано.

7. Пошаговая настройка

7.1. Подготовка пакетов

```
sudo apt update
```

Если обновление индекса не работает, проверяем NAT, DNS и маршрут до репозиториев.

7.2. Основная настройка

```
sudo apt install git -y
git config --global user.name "Student Admin"
git config --global user.email "student@example.test"
mkdir -p ~/linux-infrastructure/{configs,scripts,documentation}
cd ~/linux-infrastructure
git init
nano .gitignore
```

Содержимое `.gitignore`:

```
*.key
*.pem
*.p12
*.pcap
*.bak
*~
.env
secrets/
```

Добавляем файлы:

```
cp /usr/local/sbin/server-health.sh scripts/ 2>/dev/null || true
```

```
printf '# Linux infrastructure\n' > README.md
printf 'Учебный репозиторий без секретов.\n' > documentation/notes.md
git status
git add README.md .gitignore documentation scripts
git commit -m "Create infrastructure repository structure"
git log --oneline --decorate
```

Проверяем diff и ветку:

```
git switch -c improve-docs
printf '\nПроверяем diff перед commit.\n' >> documentation/notes.md
git diff
git add documentation/notes.md
git commit -m "Add diagnostic notes"
git switch main
git merge improve-docs
```

Самопроверка этапа

```
git status
git log --oneline --decorate --graph --all
git diff HEAD~1..HEAD
git check-ignore -v test.key
```

Ожидаем чистый working tree после commit и срабатывание `.gitignore` для секретных расширений.

Если результат отличается от ожидаемого, дальше пока не продолжаем.

Сначала проверяем этот этап.

7.3. Восстановление или откат

Перед изменением важных файлов создаём резервную копию. Для созданных сценариев и конфигов проверяем ручной запуск, код завершения и журнал. Если сценарий связан с безопасностью или backup, обязательно показываем восстановление.

8. Проверка итогового результата

```
git status
git log --oneline --decorate --graph --all
git diff HEAD~1..HEAD
```

```
git check-ignore -v test.key
```

Ожидаем чистый working tree после commit и срабатывание `.gitignore` для секретных расширений.

Границы результата:

- работа выполняется в учебной сети;
- тестовые ключи, пароли и сертификаты нельзя переносить в промышленную среду;
- успешный запуск команды не заменяет проверку результата и журналов.

9. Диагностика типовых ошибок

Симптом	Вероятная причина	Проверка	Исправление
В commit попал секрет	<code>.gitignore</code> создан поздно или файл уже staged	<code>git status</code> , <code>git diff --cached</code>	Убрать из staging и заменить файл безопасной копией

Нет автора commit	Не настроен user.name/email	<code>git config -- list</code>	Задать глобально или локально
Merge непонятен	Изменения внесены без diff	<code>git log -- graph, git diff</code>	Читать diff перед commit и писать ясные сообщения
Пакет не устанавливается	Нет доступа к репозиториям	<code>ip route, getent hosts deb.debian.org</code>	Проверить NAT и DNS
Служба не запускается	Ошибка конфигурации	<code>systemctl status, journalctl -u <service></code>	Исправить конфигурацию
Клиент не подключается	Порт не слушается или firewall блокирует	<code>ss -lntup, nc - vz <ip> <port></code>	Проверить службу и firewall

10. Итоговая самостоятельная проверка

- [] Стенд соответствует схеме.

- [] Имена и адреса согласованы.
- [] Нужные пакеты установлены.
- [] Основная настройка выполнена.
- [] Проверочные команды подтверждают результат.
- [] Клиентская проверка проходит, если сервис сетевой.
- [] В журналах нет необъяснённых ошибок.
- [] Одна типовая ошибка найдена и исправлена.

11. Что приложить к отчёту

К отчёту прикладываем схему, адресный план, ключевой фрагмент конфигурации или сценария, вывод основной проверки, клиентский результат, журнал/диагностику и описание исправленной ошибки.

12. Контрольные вопросы

1. Какую задачу решает технология этой лекции?
2. Какие исходные параметры нужно проверить до настройки?
3. Какая команда подтверждает основной результат?
4. Как отличить ошибку конфигурации от сетевой проблемы?
5. Почему нельзя хранить секреты в открытом виде?
6. Что проверить в журналах?
7. Какие ограничения учебного стенда важны?
8. Что включить в отчёт?